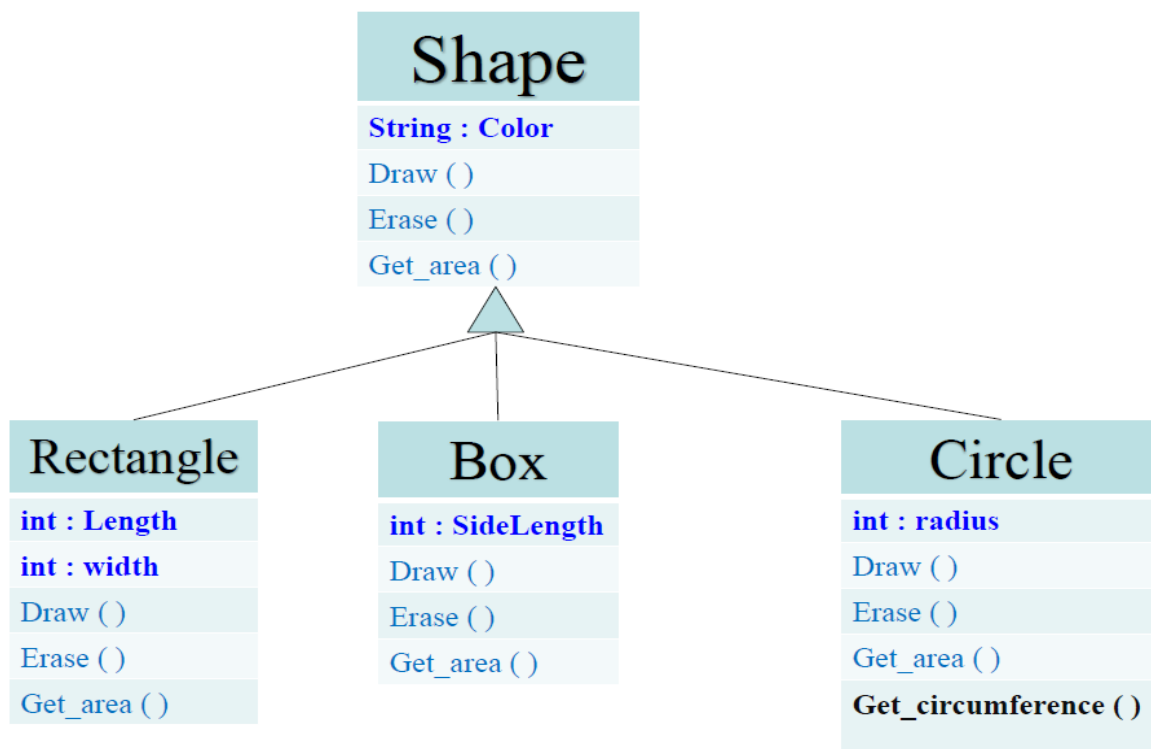


7-1 Abstract Classes (Interfaces)

- An interface (Abstract Class) describes the behavior or capabilities of a C++ class without committing to a particular implementation of that class.
- The purpose of an **abstract class** is to provide the Desired base class Form which will be inherited by other classes in the class hierarchy.
- Abstract classes cannot be used to instantiate objects and serves only as an **interface**.
- A class is made abstract by declaring at least one of its functions as **pure virtual function**.



Example:

```

#include <iostream>

using namespace std;

// Base abstract Class: Shape

class Shape {
    string color ;
public:
    Shape(string c)
    {
        color=c;
    }
    //pure virtual function
    virtual double area()=0;

    virtual void draw() =0;

    virtual void eras()=0;
};

// Derived Class: Rectangle

class Rectangle : public Shape {
private:
    double length, width;
public:
    Rectangle(double l, double w, string c): Shape(c)
    {
        length =l;
        width = w;
    }
    double area() {
        cout <<"rectangle area: " << (width * length)<<endl ;
        return (width * length);
    }
    void draw()
    {
        cout << "Drawing a Rectangle" << endl;
    }
    void eras()
    {
        cout<<"the rectangle is erased : "<<endl;
    }
};

// Derived Class: Circle
class Circle : public Shape {
private:
    double radius;

```

Abstract (interface) class used as model that summarize the work of the required system.

By doing so, we force all classes derived from the abstract class to have the same functions defined in the abstract class.

```

public:
    Circle(double r , string c) : Shape(c)
    {
        radius =r;
    }

    double area() {
        cout <<"circle area: " << 3.14 *(radius * radius) <<endl ;
        return (3.14 * radius * radius);
    }
    void draw()
    {
        cout << "Drawing a circle" << endl;
    }
    void eras()
    {
        cout<<"the Circle is erased : "<<endl;
    }
};

int main() {
    // Polymorphism: Using a base class pointer to hold derived class objects
    Shape* shap_ptr;
    Rectangle rec(10.0, 7.0, "red");
    Circle cir(5.0,"yellow");
    shap_ptr=&rec;
    shap_ptr->area();
    shap_ptr->draw();

    shap_ptr=&cir;
    shap_ptr->area();
    shap_ptr->draw();

    return 0;
}

```

```

rectangle area: 70

Drawing a Rectangle

circle area: 78.5

Drawing a circle

```

إذا حاولنا انشاء object من ال abstract class سوف تظهر رسالة خطأ لأن الكلاس معرف على انه abstract class (interface).

```

Int main ()
{ Shape S; }

```

[Error] cannot declare variable 'S' to be of abstract type 'Shape'. because the following virtual functions are pure within 'Shape':
 virtual double Shape::area()
 virtual double Shape::draw()
 virtual double Shape::Erase()

إذا حاولنا حذف أحد الدوال في الكلاسات المشتقة ولنفتراض حذفنا دالة `erase()` من الكلاس `Rectangle` سوف تظهر رسالة خطأ لأن الكلاس الأب معرف على أنه `abstract class` ونحن لم نعمل `override` على الدالة `erase()` التي اشتقت منه.

[Error] cannot declare variable 'rec' to be of abstract type 'Rectangle'
because the following virtual functions are pure within 'Rectangle':
the function `erase()` has not override.

7-2 C++ final Specifier

- ✚ Sometimes you don't want to allow derived class to override the base class virtual function. C++ 11 allows built-in facility to prevent overriding of virtual function using final specifier.

Example:

Consider the following example which shows use of final specifier. This program fails in compilation.

```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual void display()
    {
        cout<<" we are in base class "<<endl;
    }
    virtual void myfun() final
    {
        cout << "myfun() in Base display the copyright of our
company and licence we hope to read it carefully";
    }
};

class Derived : public Base
{
    void display()
    {
        cout<<" we are in derived class"<<endl;
    }
}
```

```
void myfun ()
{
    cout << "myfun() in Derived\n";
}

};

int main()
{
    Base* B_ptr;
    Derived Dev;
    B_ptr= &Dev;
    B_ptr->display();
    B_ptr->myfun();
    Base B;
    B.myfun();
}
```

[Error] overriding final function 'virtual void Base::myfun()' is prevented.

H.W:

Try to freeze the instruction of method myfun() in class Derived and see what is the change in output .

Inheritance (IS-A) vs. Composition (HAS-A) Relationship

One of the advantages of an Object-Oriented programming language is code reuse. There are two ways we can do code reuse either by the implementation of **inheritance** (IS-A relationship), or object **composition** (HAS-A relationship).

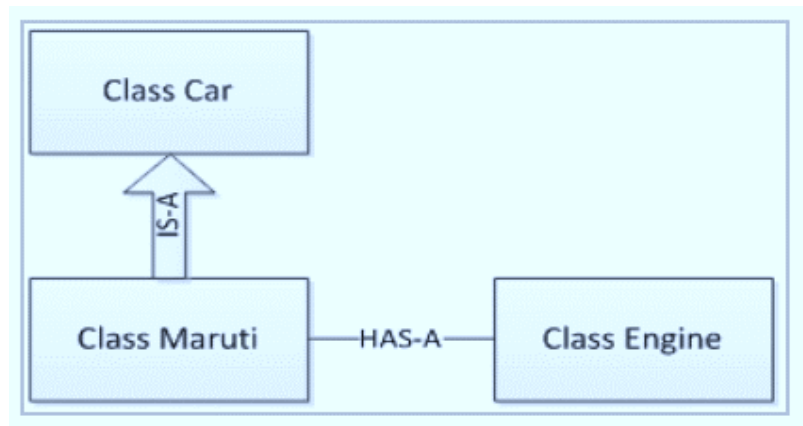
IS-A Relationship:

It is just like saying "A is a B type of thing". For example, Apple is a Fruit, Car is a Vehicle etc. Inheritance is uni-directional. For example, House is a Building. But Building is not a House.

HAS-A Relationship:

Composition(HAS-A) simply mean the use of instance variables that are references to other objects. For example, Maruti has Engine, or House has Livingroom.

Let's understand these concepts with an example of Car class.



Syntax:

```
class A
{
    // body of a class
};
class B
{
    A objA;
public:
    B(arg-list) : objA(arg-list1);
};
```

In this classes, B uses objects of class A as its data members. Hence, B is a complex class that uses a simple class A.

Example:

Write a C++ program using OOP to demonstrate the concept of Composition. Design a system for a Smartphone that consists of: **Screen**, **Battery**, and **Camera**, the, Requirements:

Create separate classes for Screen, Battery, and Camera, each with one function.

Create a class Smartphone that contains these objects (Composition).

In main(), create a Smartphone object and call its function.

Sol.

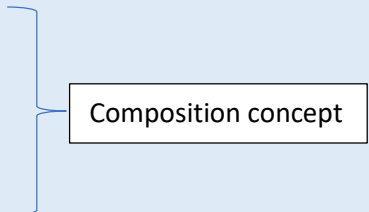
```
#include <iostream>
using namespace std;

class Screen {
public:
    void display() {
        cout << "Screen is displaying" << endl;
    }
};

class Battery {
public:
    void charge() {
        cout << "Battery charging" << endl;
    }
};

class Camera {
public:
    void takePhoto() {
        cout << "Photo taken" << endl;
    }
};

class Smartphone {
private:
    Screen screen;
    Battery battery;
    Camera camera;
public:
    void usePhone() {
        screen.display();
        battery.charge();
        camera.takePhoto();
        cout << "Using smartphone..." << endl;
    }
};
```



```
int main() {  
    Smartphone phone;  
    phone.usePhone();  
    return 0;  
}
```

Output:

Screen is displaying
Battery charging
Photo taken
Using smartphone..

H.W.

Develop the smartphone program to perform the following:

- 1- Initialize all attributes using parameterized constructors. User input:
The user should enter values for: Screen size, Battery capacity, and Camera resolution.
- 2- Use if statements: Inside each class, add conditions to evaluate performance:
for example:

```
Screen :  
If size ≥ 6 → print "Large screen"  
Else → print "Small screen"  
Battery:  
If capacity ≥ 4000 → print "Long battery life"  
Else → print "Short battery life"  
Camera:  
If resolution ≥ 12 MP → print "High quality camera"  
Else → print "Basic camera"
```

- 3- Write method called showSpecs() to display all components and their evaluations.